

Object-Oriented Programming of PHP

1. Class & Object

A class is a blueprint. An object is a real thing made from that blueprint.

Simple Example

```
<?php

class Car {                                // Define the class
    public $brand;                          // Property (variable inside class)
    public $color;

    public function drive() {              // Method (function inside class)
        return "$this->brand is driving!";
    }
}

// Create objects from the class
$car1 = new Car();
$car1->brand = "Toyota";
$car1->color = "Red";

$car2 = new Car();
$car2->brand = "Honda";

echo $car1->drive();    // Toyota is driving!
echo $car2->drive();    // Honda is driving!

?>
```

Term	Meaning
class	The blueprint / template
object	A real instance created from the class
new Car()	Creates a new object
\$this	Refers to the current object inside the class
->	Access a property or method of an object



Note: One class can create MANY objects. Each object is independent.

2. Constructor

A constructor runs automatically when an object is created. Used to set initial values.

`__construct()` Method

```
<?php

class Student {
    public $name;
    public $age;

    // Constructor – called automatically with 'new'
    public function __construct($name, $age) {
        $this->name = $name;
        $this->age = $age;
        echo "Student created: $this->name\n";
    }

    public function info() {
        return "Name: $this->name | Age: $this->age";
    }
}

// Constructor runs here automatically
$s1 = new Student("Alice", 20); // Student created: Alice
$s2 = new Student("Bob", 22); // Student created: Bob

echo $s1->info(); // Name: Alice | Age: 20
echo $s2->info(); // Name: Bob | Age: 22

?>
```

Constructor Promotion (PHP 8 Shortcut)

```
// Long way (normal):
class User {
    public string $name;
    public int $age;
    public function __construct(string $name, int $age) {
        $this->name = $name;
        $this->age = $age;
    }
}

// Short way (PHP 8 – same result!):
class User {
    public function __construct(
        public string $name,
```

```

        public int    $age
    ) {}
}

```

 **Note:** Constructor is always named `__construct()`. It has no return type.

3. Destructor

A destructor runs automatically when an object is destroyed or the script ends.

`__destruct()` Method

```

<?php

class DatabaseConnection {
    public $dbName;

    public function __construct($dbName) {
        $this->dbName = $dbName;
        echo "Connected to: $this->dbName\n";
    }

    // Destructor – runs when object is removed from memory
    public function __destruct() {
        echo "Disconnected from: $this->dbName\n";
    }
}

$db = new DatabaseConnection("myDatabase");
// ... use the database ...

// When script ends OR unset() is called:
unset($db); // Disconnected from: myDatabase

?>

```

	Constructor vs Destructor
<code>__construct()</code>	Runs when object is CREATED (new)
<code>__destruct()</code>	Runs when object is DESTROYED (unset / end of script)
Use case	Constructor = setup. Destructor = cleanup (close DB, files).

 **Note:** Destructor is rarely needed in simple apps. Useful for closing database/file connections.

4. Access Modifiers

Access modifiers control WHO can access a property or method.

Modifier	Who Can Access It
public	Everyone — inside class, outside class, child class
private	Only INSIDE the same class. Nowhere else.
protected	Inside the class AND its child (extended) classes only

Code Example

```
<?php

class BankAccount {
    public    $owner;        // Anyone can access
    protected $balance;     // Only this class + children
    private  $pin;          // Only this class

    public function __construct($owner, $balance, $pin) {
        $this->owner    = $owner;
        $this->balance = $balance;
        $this->pin      = $pin;
    }

    public function getBalance() {           // Public method to safely expose
balance    return $this->balance;
    }

    private function verifyPin($pin) {       // Private – only used internally
        return $this->pin === $pin;
    }

    public function withdraw($amount, $pin) {
        if ($this->verifyPin($pin)) {       // Can call private method inside
class        $this->balance -= $amount;
            return "Withdrawn: $amount";
        }
        return "Wrong PIN!";
    }
}

$sacc = new BankAccount("Alice", 1000, 1234);

echo $sacc->owner;           // Alice           ✓ public
echo $sacc->getBalance();    // 1000           ✓ public method
```

```

echo $acc->withdraw(200, 1234); // Withdrawn: 200

// echo $acc->pin;           // ✗ ERROR – private
// echo $acc->balance;       // ✗ ERROR – protected

?>

```

Getters & Setters (Best Practice for private)

```

class Person {
    private string $name;

    public function getName(): string { // Getter
        return $this->name;
    }

    public function setName(string $n): void { // Setter
        if (strlen($n) > 0) {
            $this->name = $n;
        }
    }
}

$p = new Person();
$p->setName("John");
echo $p->getName(); // John

```

5. Inheritance

Inheritance lets a child class REUSE properties and methods from a parent class.

Keyword used: extends

Basic Inheritance

```

<?php

// PARENT class
class Animal {
    public $name;

    public function __construct($name) {
        $this->name = $name;
    }

    public function eat() {
        return "$this->name is eating.";
    }
}

```

```

        public function sleep() {
            return "$this->name is sleeping.";
        }
    }

    // CHILD class – inherits from Animal
    class Dog extends Animal {
        public function bark() {
            return "$this->name says: Woof!";
        }
    }

    class Cat extends Animal {
        public function meow() {
            return "$this->name says: Meow!";
        }
    }

    $dog = new Dog("Rex");
    echo $dog->eat();      // Rex is eating.    (from Animal)
    echo $dog->bark();     // Rex says: Woof!   (from Dog)

    $cat = new Cat("Kitty");
    echo $cat->sleep();    // Kitty is sleeping. (from Animal)
    echo $cat->meow();     // Kitty says: Meow!  (from Cat)

    ?>

```

Using parent:: (Call Parent Method)

```

class Vehicle {
    public function __construct(public string $brand) {
        echo "Vehicle created: $brand\n";
    }
}

class Car extends Vehicle {
    public string $model;

    public function __construct(string $brand, string $model) {
        parent::__construct($brand); // Call parent constructor
        $this->model = $model;
        echo "Car model: $model\n";
    }
}

$c = new Car("Toyota", "Corolla");
// Vehicle created: Toyota
// Car model: Corolla

```

protected in Inheritance

```
class Animal {
    protected $sound = "...";    // Child classes CAN access

    public function makeSound() {
        return $this->sound;
    }
}

class Dog extends Animal {
    public function __construct() {
        $this->sound = "Woof";    // ✅ Can access protected
    }
}

$d = new Dog();
echo $d->makeSound();    // Woof
```

💡 **Note:** private = child CANNOT access. protected = child CAN access. public = everyone can access.

6. Polymorphism

Poly = many. Morph = forms. Same method name, different behavior in different classes.

Method Overriding (Runtime Polymorphism)

Child class redefines a method from the parent class.

```
<?php

class Shape {
    public function area() {
        return "Area not defined.";
    }
}

class Circle extends Shape {
    public function __construct(private float $radius) {}

    // OVERRIDE parent's area()
    public function area() {
        return 3.14 * $this->radius ** 2;
    }
}

class Rectangle extends Shape {
    public function __construct(
        private float $width,
        private float $height
    ) {}

    // OVERRIDE parent's area()
    public function area() {
        return $this->width * $this->height;
    }
}

$shapes = [
    new Circle(5),
    new Rectangle(4, 6),
    new Shape()
];

foreach ($shapes as $shape) {
    echo $shape->area() . "\n";
}
// 78.5
// 24
// Area not defined.

?>
```


Abstract Class (Force child to implement method)

```
abstract class Animal {
    abstract public function sound(); // No body – child MUST define it

    public function breathe() {        // Normal method – inherited
        return "Breathing...";
    }
}

class Dog extends Animal {
    public function sound() {           // MUST implement abstract method
        return "Woof!";
    }
}

class Cat extends Animal {
    public function sound() {
        return "Meow!";
    }
}

// $a = new Animal(); // ❌ ERROR – cannot instantiate abstract class

$dog = new Dog();
echo $dog->sound();    // Woof!
echo $dog->breathe();  // Breathing...
```

Interface (Pure Polymorphism Contract)

```
interface Drawable {
    public function draw();    // No body – just the rule
}

interface Resizable {
    public function resize(float $factor);
}

// A class can implement MULTIPLE interfaces
class Circle implements Drawable, Resizable {
    public function __construct(private float $radius) {}

    public function draw() {
        return "Drawing circle with radius $this->radius";
    }

    public function resize(float $factor) {
        $this->radius *= $factor;
    }
}
```

```
}
```

```
$c = new Circle(5);  
echo $c->draw();    // Drawing circle with radius 5  
$c->resize(2);  
echo $c->draw();    // Drawing circle with radius 10
```

	Abstract Class vs Interface
Can have normal methods?	Abstract:  YES Interface:  NO (only method signatures)
Can have properties?	Abstract:  YES Interface:  NO
Multiple inheritance?	Abstract:  One only Interface:  Many allowed
Keyword	Abstract: extends Interface: implements

7. Full OOP Example (All Concepts Together)

```
<?php

// Abstract parent class
abstract class Employee {
    protected string $name;
    protected float $baseSalary;

    public function __construct(string $name, float $salary) {
        $this->name = $name;
        $this->baseSalary = $salary;
        echo "Hired: $this->name\n";
    }

    // Abstract – every child must define this
    abstract public function calculatePay(): float;

    // Normal shared method
    public function getInfo(): string {
        return "$this->name – Pay: $" . $this->calculatePay();
    }

    public function __destruct() {
        echo "$this->name record cleared.\n";
    }
}

// Child 1
class FullTimeEmployee extends Employee {
    public function calculatePay(): float {
        return $this->baseSalary; // Fixed salary
    }
}

// Child 2
class PartTimeEmployee extends Employee {
    private int $hoursWorked;

    public function __construct(string $name, float $rate, int $hours) {
        parent::__construct($name, $rate);
        $this->hoursWorked = $hours;
    }

    public function calculatePay(): float {
        return $this->baseSalary * $this->hoursWorked; // Hourly
    }
}

$employees = [
    new FullTimeEmployee("Alice", 3000),
```

```
        new PartTimeEmployee("Bob", 15, 80),
    ];

// Polymorphism – same getInfo(), different calculatePay()
foreach ($employees as $emp) {
    echo $emp->getInfo() . "\n";
}
// Alice – Pay: $3000
// Bob   – Pay: $1200

?>
```

8. OOP Quick Cheat Sheet

Concept	Key Point
class	Blueprint / template for objects
object	Instance created with new ClassName()
\$this	Refers to current object inside the class
__construct()	Auto-runs when object is created. Used for setup.
__destruct()	Auto-runs when object is destroyed. Used for cleanup.
public	Accessible from everywhere
private	Accessible only inside the same class
protected	Accessible inside class + child classes
extends	Child inherits from parent class
parent::	Call parent's constructor or method from child
override	Child redefines parent method with same name
abstract class	Can have abstract methods. Cannot be instantiated.
interface	Only method signatures. A class implements many interfaces.
Polymorphism	Same method name, different behavior per class

4 Pillars of OOP

- Encapsulation — Hide data using private/protected. Expose via public methods.
- Inheritance — Child class reuses code from parent using extends.
- Polymorphism — Same method, different behavior in different classes.
- Abstraction — Hide complexity. Show only what's needed (abstract / interface).